

5.3 Code/Program

ga-ann.py

```
import csv
import random

import joblib
import numpy as np
import pandas as pd
import shap
from imblearn.over_sampling import SMOTE
from silence_tensorflow import silence_tensorflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import mutual_info_classif
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, minmax_scale

silence_tensorflow()

import tensorflow as tf # noqa: F401,E402
from deap import algorithms, base, creator, tools # noqa: E402
from keras.backend import clear_session # noqa: E402
from keras.callbacks import EarlyStopping, ReduceLROnPlateau # noqa: E402
from keras.layers import ( # noqa: F401,E402,E501 # noqa: E402 # noqa: E402
    ELU,
    Activation,
    BatchNormalization,
    Dense,
    Dropout,
    InputLayer,
    LeakyReLU,
    PReLU,
    ReLU,
```

```

)
from keras.metrics import AUC # noqa: E402
from keras.models import Sequential # noqa: E402
from keras.optimizers import Adam # noqa: E402
from keras.regularizers import l2 # noqa: E402

# Hyperparameter Ranges
hyprparameter_ranges = {
    "l1": (128, 192),
    "l2": (64, 128),
    "l3": (32, 64),
    "learning_rate": (0.0001, 0.01),
    "dropout_rate": (0.0, 0.4),
    "l2_regularization": (0.000001, 0.01),
    "alpha": (0.01, 0.3),
}

# GA Parameters
ga_parameters = {"generation": 2, "population": 5}

# Load dataset
data = pd.read_csv("heart_dataset.csv")
feature_names = data.columns[data.columns != "target"].tolist()
input_shape = len(feature_names)

X = data.iloc[:, :input_shape].values
y = data["target"].values

# Oversample using SMOTE
X, y = SMOTE(random_state=42).fit_resample(X, y)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

```

```

# Scaling
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Dump scaler # We need this to use pre-trained model
# pre-trained model require same scaler as training scaler
joblib.dump(sc, "models/scaler.pkl")

# Compute feature importance (RF + MI + Shap)
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)

explainer = shap.Explainer(rf)
shap_values = explainer(X_train, check_additivity=False)
shap_values_class1 = shap_values.values[:, :, 1]

rf_importance = rf.feature_importances_
mi_importance = mutual_info_classif(X_train, y_train)
shap_importance = np.abs(shap_values_class1).mean(axis=0)

combined_score = (
    minmax_scale(rf_importance)
    + minmax_scale(mi_importance)
    + minmax_scale(shap_importance)
) / 3
ranked_features = np.argsort(combined_score)[::-1]

# Logging
log_file = "models/logs/ReLU_model.csv"
with open(log_file, mode="w", newline="") as f:
    writer = csv.writer(f)
    header = ["Generation", "SelectedFeatures"]
    for key in hyperparameter_ranges:

```

```
    header.append(key)
    header.append("Accuracy")
    writer.writerow(header)
```

```
# Early stopping
early_stopping = EarlyStopping(
    start_from_epoch=25, monitor="val_loss", min_delta=0.001, patience=5
)
```

```
# Reduce Learning
reduce_lr = ReduceLROnPlateau(monitor="val_loss", min_lr=0.0001, factor=0.8,
    patience=2)
```

```
# ANN Model
def create_ann_model(hyprparameters, input_shape):
    layer_keys = []
```

```
    for key in hyprparameter_ranges:
        if key.startswith("l") and key[1:].isdigit():
            layer_keys.append(key)
```

```
    hidden_layers = len(layer_keys)
```

```
    clear_session()
    layer_units = hyprparameters[:hidden_layers]
    lr, dr, l2_reg, alpha = hyprparameters[hidden_layers:]
```

```
    model = Sequential()
    model.add(InputLayer(shape=(input_shape,)))
```

```
# Hidden Layers
for neurons in layer_units:
    model.add(Dense(int(neurons), kernel_regularizer=l2(l2_reg)))
    model.add(ReLU())
```

```

        model.add(BatchNormalization())
        model.add(Dropout(dr))

    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss="binary_crossentropy",
        metrics=["accuracy", AUC(name="auc")],
    )

    model_history = model.fit(
        X_train_selected,
        y_train,
        validation_split=0.30,
        epochs=145,
        batch_size=35,
        callbacks=[early_stopping, reduce_lr],
        verbose=0,
    )

    return model, model_history

```

Hyprparameter Generator

```

def random_hyprparameters(hyprparameter_ranges):
    hyprparameter = []
    for key, (low, high) in hyprparameter_ranges.items():
        if isinstance(low, int) and isinstance(high, int):
            random_value = random.randint(low, high)
        elif isinstance(low, float) and isinstance(high, float):
            random_value = random.uniform(low, high)

    hyprparameter.append(random_value)

```

```
return hyprparameter
```

```
# Feature Selection using top-K ranked features + Random Hyprparamaters
```

```
def smart_individual(ranked_features, hyprparameter_ranges, input_shape):
```

```
    k = random.randint(6, 10)
```

```
    feature_mask = [0] * 12
```

```
    for idx in ranked_features[:k]:
```

```
        feature_mask[idx] = 1
```

```
    hyprparameter = random_hyprparameters(hyprparameter_ranges)
```

```
    return feature_mask + hyprparameter
```

```
# Evaluation
```

```
def evaluate(individual, input_shape, generation):
```

```
    feature_mask = individual[:input_shape]
```

```
    hyprparameters = individual[input_shape:]
```

```
    selected_indices = []
```

```
    selected_features = []
```

```
    for i, bit in enumerate(feature_mask):
```

```
        if bit == 1:
```

```
            selected_indices.append(i)
```

```
    for i, bit in enumerate(feature_mask):
```

```
        if bit == 1:
```

```
            selected_features.append(feature_names[i])
```

```
    selected_input_shape = len(selected_indices)
```

```
    global X_train_selected, X_test_selected
```

```
    X_train_selected = X_train[:, selected_indices]
```

```
X_test_selected = X_test[:, selected_indices]
```

```
model, _ = create_ann_model(hyprparameters, selected_input_shape)
```

```
y_probability = model.predict(X_test_selected)
```

```
y_prediction = y_probability > 0.5
```

```
accuracy = accuracy_score(y_test, y_prediction)
```

```
with open(log_file, mode="a", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    log = [generation, ",".join(selected_features)]
```

```
    for parameter in hyprparameters:
```

```
        log.append(parameter)
```

```
    log.append(f"{accuracy * 100:.4f} %")
```

```
    writer.writerow(log)
```

```
return (accuracy,)
```

```
# Repair function
```

```
def repair(individual, ranked_features, input_shape):
```

```
    hypr_shape = input_shape + len(hyprparameter_ranges)
```

```
    feature_mask = individual[:input_shape]
```

```
    hyprparameter = individual[input_shape:hypr_shape]
```

```
# Feature Selection
```

```
for i in range(input_shape):
```

```
    if feature_mask[i] in (0, 1):
```

```
        feature_mask[i] = int(individual[i])
```

```
    else:
```

```
        feature_mask[i] = 1
```

```
if sum(feature_mask[:input_shape]) == 0:
```

```
    k = random.randint(6, 10)
```

```

    for idx in ranked_features[:k]:
        feature_mask[idx] = 1

# Hyprparameters
for i, (key, (low, high)) in enumerate(hyprparameter_ranges.items()):
    clipped = np.clip(hyprparameter[i], low, high)

    if isinstance(low, int) and isinstance(high, int):
        hyprparameter[i] = int(clipped)
    elif isinstance(low, float) and isinstance(high, float):
        hyprparameter[i] = float(clipped)

individual[:input_shape] = feature_mask
individual[input_shape:hypr_shape] = hyprparameter

return individual

# GA setup
toolbox = base.Toolbox()

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox.register(
    "individual",
    tools.initIterate,
    creator.Individual,
    lambda: smart_individual(ranked_features, hyprparameter_ranges, input_shape),
)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

```



```

toolbox.register("evaluate", evaluate)

# Run GA
generations = ga_parameters["generation"]
population_size = ga_parameters["population"]
best_accuracies = []

population = toolbox.population(n=population_size)

print("\n...Starting Genetic Algorithm Optimization...\n")

for generation in range(1, generations + 1):
    print(f"=== Generation {generation} ===")

    offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.2)
    for i, individual in enumerate(offspring):
        offspring[i] = repair(individual, ranked_features, input_shape)

    for individuals in offspring:
        individuals.fitness.values = evaluate(individuals, input_shape, generation)

    population = toolbox.select(offspring, k=len(population))
    best_individual = tools.selBest(population, k=1)[0]
    best_accuracies.append(best_individual.fitness.values[0])

    print("Best individual so far:", best_individual)
    print(f"Best accuracy: {best_individual.fitness.values[0] * 100:.2f}%")

# Final model training
print("Training model with best parameters...")
best_individual = tools.selBest(population, k=1)[0]
hypr_shape = input_shape + len(hyprparameter_ranges)

feature_mask = best_individual[:input_shape]
hyprparameters = best_individual[input_shape:hypr_shape]

```

```

selected_indices = []

for i, bit in enumerate(feature_mask):
    if bit == 1:
        selected_indices.append(i)

X_train_selected = X_train[:, selected_indices]
X_test_selected = X_test[:, selected_indices]
selected_input_shape = len(selected_indices)

model, model_history = create_ann_model(hyprparameters, selected_input_shape)

model.save("models/ReLU_heart_model.keras")

log_data = {
    "history": model_history.history,
    "selected_features": selected_indices,
    "hyprparameters": hyprparameters,
}
joblib.dump(log_data, "models/logs/ReLU_model_logs.pkl")

# Final evaluation
y_probability = model.predict(X_test_selected)
y_prediction = (y_probability > 0.5).astype(int)
accuracy = accuracy_score(y_test, y_prediction)
print(f"Final Test Accuracy: {accuracy * 100:.2f}%")

```

model_analysis.ipynb

```

from silence_tensorflow import silence_tensorflow
silence_tensorflow()

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

```

```

# Inline Plots in Jupyter
%matplotlib inline

models = ['ReLU', 'LeakyReLU', 'PReLU', 'ELU', 'SELU', 'Tanh']

# Load Modelfiles
from keras.models import load_model

base_path="./models/"
suffix="_heart_model.keras"

for model in models:
    modelfile = f"{base_path}{model}{suffix}"
    model_var = f"{model}_model"
    globals()[model_var] = load_model(modelfile)
    print(f" - {model_var}: {'loaded' if model_var in globals() else 'missing'}")

# Load Model logfiles
import joblib

base_path="./models/logs/"
suffix="_model_logs.pkl"

for model in models:
    logfile = f"{base_path}{model}{suffix}"
    log_var = f"{model}_logs"
    globals()[log_var] = joblib.load(logfile)
    print(f" - {log_var}: {'loaded' if log_var in globals() else 'missing'}")

# Extract History
for model in models:
    log_var = f"{model}_logs"
    hist_var = f"{model}_history"

    log_obj = globals().get(log_var)

```

```

if isinstance(log_obj, dict):
    history = log_obj.get('history')
    globals()[hist_var] = history

print(f" - {hist_var}: {'loaded' if hist_var in globals() else 'missing'}")

print(f"Model Saved Paramaters {list(ReLU_history.keys())}")

data = pd.read_csv("../heart_dataset.csv")
feature_names = list(data.columns[:12])
X = data.iloc[:, :12].values
y = data["target"].values
data.shape

# import scaler and scale our test instance
# the test data and pre-train model should be scaled by same scalar
sc = joblib.load("../models/scaler.pkl")
X_scaled = sc.transform(X)

def show_model_info(model, logs):
    history = logs['history']
    params = logs['hyperparams']
    sel = logs['selected_features']

    sel_indices = list(sel)
    col_names = [feature_names[i] for i in sel_indices]

    X_sel = pd.DataFrame(X_scaled).iloc[:, sel_indices]
    X_sel.columns = col_names

    print(f"=== {model} ===")
    print("Model Parameters:")
    print(f"Number of epochs: {len(history.get('accuracy', []))}")
    print(f"1st Hidden Layer (n1): {params[0]} neurons")
    print(f"2nd Hidden Layer (n2): {params[1]} neurons")

```

```
print(f"3rd Hidden Layer (n3): {params[2]} neurons")
print(f"Learning Rate (lr): {params[3]}")
print(f"Dropout Rate (dr): {params[4]}")
print(f"L2 Regularization (l2): {params[5]}")
print("Selected features indices:", sel_indices)
print("Selected feature names:", col_names)
print()
```

```
for model in models:
```

```
    model_log = globals().get(f"{model}_logs")
    show_model_info(model, model_log)
```

```
for model in models:
```

```
    var = globals().get(f"{model}_model")
    print(f"=== {model} ===")
    var.summary()
```

```
def plot_stats(ax, model, metric):
```

```
    model_history = globals().get(f"{model}_history")
    train_metric = model_history.get(metric)
    val_metric = model_history.get(f'val_{metric}')
```

```
    if train_metric is None or val_metric is None:
```

```
        ax.set_title(f"{model}: '{metric}' not found")
        ax.axis('off')
        return
```

```
epochs = range(1, len(train_metric) + 1) # Epochs starting from 1
```

```
ax.plot(epochs, train_metric, label='Train')
ax.plot(epochs, val_metric, label='Validation')
ax.set_title(f'{model}')
ax.set_xlabel('Epochs')
ax.set_ylabel(metric.capitalize())
ax.legend()
```

```
ax.grid(True)
```

```
metric = 'accuracy'
```

```
fig, axes = plt.subplots(2, 3, figsize=(5 * 4, 4 * 3))  
axes = axes.flatten()
```

```
for i, model in enumerate(models):  
    plot_stats(axes[i], model, metric)
```

```
plt.tight_layout()  
plt.show()
```

```
metric = 'loss'
```

```
fig, axes = plt.subplots(2, 3, figsize=(5 * 4, 4 * 3))  
axes = axes.flatten()
```

```
for i, model in enumerate(models):  
    plot_stats(axes[i], model, metric)
```

```
plt.tight_layout()  
plt.show()
```

```
metric = 'auc'
```

```
fig, axes = plt.subplots(2, 3, figsize=(5 * 4, 4 * 3))  
axes = axes.flatten()
```

```
for i, model in enumerate(models):  
    plot_stats(axes[i], model, metric)
```

```
plt.tight_layout()  
plt.show()
```

```
metric = 'learning_rate'
```

```

def plot_stats(ax, model):
    model_history = globals().get(f"{model}_history")
    train_metric = model_history.get('learning_rate')

    if train_metric is None:
        ax.set_title(f"{model}: '{metric}' not found")
        ax.axis('off')
        return

    ax.plot(train_metric, label='Learning Rate')
    ax.set_title(f'{model} - Model Learning Rate')
    ax.set_xlabel('Epochs')
    ax.set_ylabel(metric.capitalize())
    ax.legend()
    ax.grid(True)

fig, axes = plt.subplots(2, 3, figsize=(5 * 4, 4 * 3))
axes = axes.flatten()

for i, model in enumerate(models):
    plot_stats(axes[i], model)

plt.tight_layout()
plt.show()

for model in models:
    logs = globals().get(f"{model}_logs")
    sel = logs.get('selected_features')
    selected_indices = list(sel)

    X_model = X_scaled[:, selected_indices]
    globals()[f"X_{model}"] = X_model
    print(f" - X_{model}: {'loaded' if f'X_{model}' in globals() else 'missing'}")

```

```

from sklearn.metrics import accuracy_score

def model_accuracy(model_name):
    model = globals().get(f"{model_name}_model")
    X_data = globals().get(f"X_{model_name}")

    model_prob = model.predict(X_data)
    model_pred = (model_prob > 0.5).astype(int).flatten()

    acc = accuracy_score(y, model_pred)
    print(f"{model_name} Accuracy: {round(acc*100, 2)}%")

for model in models:
    model_accuracy(model)

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def plot_confusion_matrix(ax, model_name):
    model = globals().get(f"{model_name}_model")
    X_data = globals().get(f"X_{model_name}")

    model_prob = model.predict(X_data, verbose=0)
    model_pred = (model_prob > 0.5).astype(int).flatten()

    cm = confusion_matrix(y, model_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot(ax=ax, cmap='magma', colorbar=False)
    ax.set_title(f"{model_name}")

fig, axes = plt.subplots(2, 3, figsize=(5 * 4, 4 * 3))
axes = axes.flatten()

for i, model in enumerate(models):

```



```
plot_confusion_matrix(axes[i], model)
```

```
plt.tight_layout()
```

```
plt.show()
```